

An Even More Annotated Transformer

Architecture, Design Choices, Implementation, and Inference

Typeset and edited from the supplied manuscript

13 July 2023

Abstract

This paper develops an implementation-oriented account of the Transformer architecture. It begins with scaled dot-product attention, interprets attention as a differentiable key–value lookup, and then derives multi-head attention. It proceeds through pre-layer-normalized encoder and decoder blocks, token and positional embeddings, weight tying, complete encoder–decoder assembly, teacher-forced training, and autoregressive inference. A compact PyTorch implementation is presented together with two experiments: sequence reversal and German–English machine translation. The final section discusses batching examples by length to reduce padding overhead.

Contents

1	Introduction	2
2	Attention	2
2.1	Contextualizing independent encodings	2
2.2	Queries, keys, and values	2
2.3	Attention as a soft key–value lookup	3
2.4	Why divide by $\sqrt{d_k}$?	3
2.5	Masking	3
3	Multi-Head Attention	4
4	Encoder Block	5
5	Decoder Block	7
6	Token and Position Embeddings	9
7	Complete Transformer	10
8	Autoregressive Inference	12
9	Experiment I: Reversing a Sequence	13
10	Experiment II: German–English Translation	15
11	Batching by Sequence Length	17
12	Discussion	17
13	Conclusion	18

1 Introduction

The Transformer replaces sequential recurrent computation with attention-based interactions that can be evaluated in parallel across positions. The original architecture was introduced by Vaswani et al. [1]; the presentation here follows the implementation-oriented tradition of the *Annotated Transformer* [2], while adding explicit discussion of dimensionality, variance, residual pathways, masking, normalization placement, and practical data handling.

Let an input sequence contain T elements,

$$X = \begin{bmatrix} x_1^\top & x_2^\top & \cdots & x_T^\top \end{bmatrix}^\top \in \mathbb{R}^{T \times d_{\text{model}}}.$$

The central goal is to map each x_i to a contextual representation z_i that depends on the other sequence elements while avoiding the strictly sequential unrolling required by a recurrent neural network.

2 Attention

2.1 Contextualizing independent encodings

A simple linear encoding Wx_i can be evaluated independently for every position, but it does not couple position i to the rest of the sequence. Self-attention introduces such coupling by computing a value vector for every element and then taking a data-dependent weighted average:

$$v_j = x_j W_V, \tag{1}$$

$$z_i = \sum_{j=1}^T \alpha_{ij} v_j, \quad \sum_{j=1}^T \alpha_{ij} = 1, \quad \alpha_{ij} \geq 0. \tag{2}$$

The weight α_{ij} should be large when element i considers element j relevant.

2.2 Queries, keys, and values

A symmetric score such as $x_i^\top x_j$ cannot distinguish “ i attends to j ” from “ j attends to i .” The Transformer therefore learns separate query and key projections:

$$q_i = x_i W_Q \in \mathbb{R}^{d_k}, \tag{3}$$

$$k_j = x_j W_K \in \mathbb{R}^{d_k}, \tag{4}$$

$$v_j = x_j W_V \in \mathbb{R}^{d_v}. \tag{5}$$

The unnormalized compatibility score is

$$s_{ij} = \frac{q_i k_j^\top}{\sqrt{d_k}}, \tag{6}$$

and the attention weights are obtained row-wise by softmax:

$$\alpha_{ij} = \frac{\exp(s_{ij})}{\sum_{m=1}^T \exp(s_{im})}. \tag{7}$$

In matrix form,

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right) V. \tag{8}$$

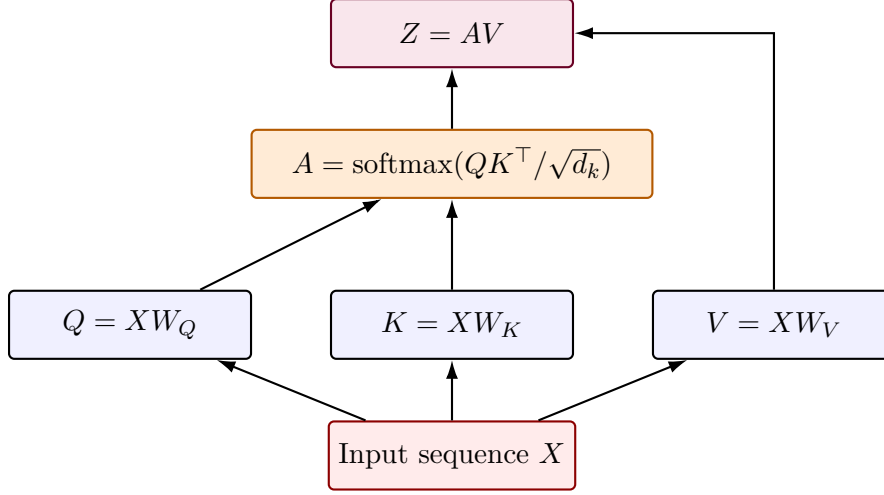


Figure 1: Scaled dot-product self-attention. All three projections are constructed from the same sequence in self-attention.

2.3 Attention as a soft key–value lookup

Equation (8) can be interpreted as a differentiable lookup table. A conventional database matches a query to one discrete key and returns its associated value. Attention matches a query softly against every key, producing weights between zero and one, and returns the corresponding weighted combination of values. In self-attention, the key–value store is constructed from the same sequence being queried. In cross-attention, the keys and values come from an encoded source sequence, whereas the queries come from the target sequence.

2.4 Why divide by $\sqrt{d_k}$?

Assume that the components of q and k are independent, zero mean, and unit variance. Then

$$qk^\top = \sum_{r=1}^{d_k} q_r k_r, \quad \text{Var}(qk^\top) = d_k. \quad (9)$$

Thus the standard deviation of the raw dot product grows like $\sqrt{d_k}$. Large-magnitude logits make softmax nearly one-hot and produce small gradients for most entries. Dividing by $\sqrt{d_k}$ approximately restores unit-scale logits under this idealized initialization model.

2.5 Masking

Attention masks serve two common purposes:

- (a) **Padding masks** remove padded sequence positions from the key–value store.
- (b) **Causal masks** prevent target position i from attending to future positions $j > i$.

Operationally, forbidden logits are replaced by a large negative value before softmax:

$$S_{ij} \leftarrow -\infty \quad \text{whenever position } j \text{ is masked for query } i. \quad (10)$$

After softmax, the corresponding probability is zero.

3 Multi-Head Attention

A single attention distribution may concentrate on one dominant relation. Multi-head attention supplies H independent query, key, and value projections so that different heads can represent distinct relations or subspaces:

$$\text{head}_h = \text{Attention}(XW_Q^{(h)}, XW_K^{(h)}, XW_V^{(h)}), \quad (11)$$

$$\text{MHA}(X) = \text{Concat}(\text{head}_1, \dots, \text{head}_H)W_O. \quad (12)$$

Typically $d_k = d_v = d_{\text{model}}/H$, so concatenating all heads restores dimension d_{model} and avoids multiplying the leading parameter count by H .

Factoring the score through W_Q and W_K is also useful beyond the query-key metaphor. If the score were written directly as $x_i M x_j^\top$, then M would be a full $d_{\text{model}} \times d_{\text{model}}$ matrix. Writing $M = W_Q W_K^\top$ with $W_Q, W_K \in \mathbb{R}^{d_{\text{model}} \times d_k}$ constrains $\text{rank}(M) \leq d_k$ and permits a lower-dimensional matching space.

```

1 class MultiHeadAttention(nn.Module):
2     def __init__(self, in_dim, qk_dim, v_dim, out_dim,
3         n_heads, attn_dropout=0.0):
4         super().__init__()
5         self.n_heads = n_heads
6         self.Q = nn.Linear(in_dim, qk_dim, bias=False)
7         self.K = nn.Linear(in_dim, qk_dim, bias=False)
8         self.V = nn.Linear(in_dim, v_dim, bias=False)
9         self.Wo = nn.Linear(v_dim, out_dim)
10        self.attn_dropout = nn.Dropout(attn_dropout)
11
12        head_qk = qk_dim // n_heads
13        nn.init.normal_(
14            self.Q.weight,
15            std=np.sqrt(2.0 / (in_dim + head_qk)),
16        )
17        nn.init.normal_(
18            self.K.weight,
19            std=np.sqrt(2.0 / (in_dim + head_qk)),
20        )
21        nn.init.zeros_(self.Wo.bias)

```

Listing 1: Multi-head attention layer.

The forward pass projects the three inputs, reshapes them into heads, computes scaled attention scores, applies an optional mask and dropout, concatenates the per-head outputs, and applies the output projection.

```

1 class MultiHeadAttention(nn.Module):
2     # __init__ as above
3
4     def forward(self, queries, keys, values, mask=None):
5         B, T, _ = queries.shape
6         _, Ts, _ = keys.shape
7
8         if mask is not None:
9             mask = mask.unsqueeze(dim=1) # account for head dim
10
11        q = self.Q(queries).view(B, T, self.n_heads, -1)
12        q = q.transpose(1, 2)
13        k = self.K(keys).view(B, Ts, self.n_heads, -1)
14        k = k.transpose(1, 2)
15        v = self.V(values).view(B, Ts, self.n_heads, -1)
16        v = v.transpose(1, 2)
17

```

```

18     scores = torch.matmul(q, k.transpose(2, 3))
19     scores = scores / np.sqrt(k.shape[-1])
20
21     if mask is not None:
22         scores.masked_fill_(~mask, -1e9)
23
24     attn = torch.softmax(scores, dim=-1)
25     attn = self.attn_dropout(attn)
26
27     z = torch.matmul(attn, v)
28     z = z.transpose(1, 2).reshape(B, T, -1)
29     out = self.Wo(z)
30     return out, attn

```

Listing 2: Forward pass of the multi-head attention layer.

Self-attention is invoked as

```

1 z, attn = attn_layer(x, x, x)

```

whereas cross-attention passes a different sequence for keys and values.

Dropout is often applied directly to the attention probabilities during training. The retained entries no longer sum exactly to one in a particular dropout realization, but the operation is disabled during evaluation and its expectation is controlled by standard inverted dropout scaling.

4 Encoder Block

An encoder consists of N_{enc} repeated blocks. Each block contains self-attention and a position-wise feed-forward network. The implementation considered here uses *Pre-LayerNorm*:

$$U = X + \text{Dropout}(\text{MHA}(\text{LayerNorm}(X))), \quad (13)$$

$$Y = U + \text{Dropout}(\text{FFN}(\text{LayerNorm}(U))), \quad (14)$$

with

$$\text{FFN}(x) = W_2 \sigma(W_1 x + b_1) + b_2. \quad (15)$$

The inner dimension is usually several times d_{model} ; the original Transformer used $4d_{\text{model}}$.

Residual connections make it possible for deeper stacks to approximate shallower ones by learning small residual updates. They also preserve a direct pathway for the initial token-plus-position representation. The feed-forward network contributes an element-wise nonlinear transformation; attention alone mainly forms data-dependent linear combinations of value vectors.

```

1 class EncoderBlock(nn.Module):
2     def __init__(self, d_model, n_heads, dim_mlp, dropout):
3         super().__init__()
4         self.attn = MultiHeadAttention(
5             in_dim=d_model,
6             qk_dim=d_model,
7             v_dim=d_model,
8             out_dim=d_model,
9             n_heads=n_heads,
10            attn_dropout=dropout,
11        )
12        self.attn_dropout = nn.Dropout(dropout)
13        self.attn_norm = nn.LayerNorm(d_model)
14
15        self.mlp = nn.Sequential(
16            nn.Linear(d_model, dim_mlp),
17            nn.ReLU(),

```

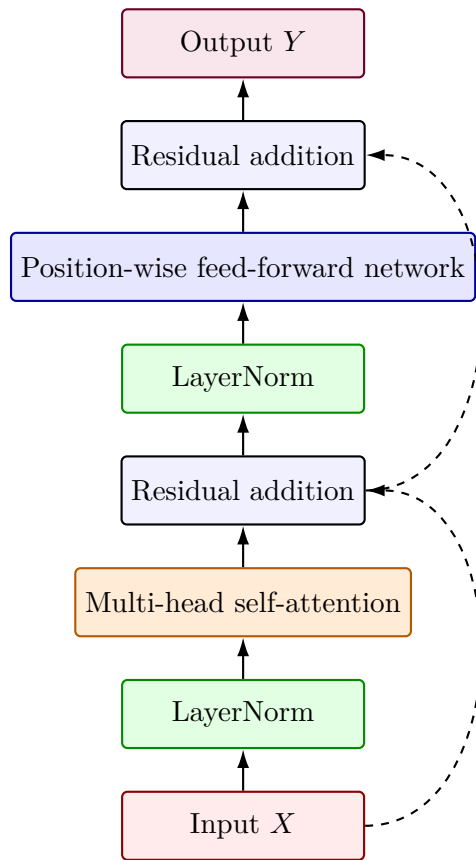


Figure 2: Pre-LayerNorm encoder block. A final LayerNorm is applied after the complete encoder stack.

```

18         nn.Linear(dim_mlp, d_model),
19     )
20     self.mlp_dropout = nn.Dropout(dropout)
21     self.mlp_norm = nn.LayerNorm(d_model)
22
23     def forward(self, x, mask=None):
24         attn_input = self.attn_norm(x)
25         z, _ = self.attn(attn_input, attn_input,
26                         attn_input, mask=mask)
27         z = x + self.attn_dropout(z)
28
29         r = self.mlp(self.mlp_norm(z))
30         r = z + self.mlp_dropout(r)
31         return r

```

Listing 3: Pre-LayerNorm encoder block.

The original Transformer used post-normalization, placing LayerNorm after each residual addition. Pre-LayerNorm places it before each sublayer and often improves optimization stability in deep models [4]. Because the residual stream is not normalized after each block, a final LayerNorm is added after the stack.

5 Decoder Block

The decoder adds two mechanisms to the encoder pattern:

1. masked self-attention over the target prefix;
2. cross-attention from the target representation to encoder memory.

For encoder memory M , a Pre-LayerNorm decoder block can be written as

$$U = X + \text{Dropout}(\text{MHA}_{\text{causal}}(\text{LayerNorm}(X))), \quad (16)$$

$$C = U + \text{Dropout}(\text{CrossAttn}(\text{LayerNorm}(U), M, M)), \quad (17)$$

$$Y = C + \text{Dropout}(\text{FFN}(\text{LayerNorm}(C))). \quad (18)$$

In cross-attention, queries come from the target stream, whereas keys and values come from the encoder output. For target length T_{tgt} and source length T_{src} , each head produces an attention-score matrix of shape $T_{\text{tgt}} \times T_{\text{src}}$.

```

1 class DecoderBlock(nn.Module):
2     def __init__(self, d_model, n_heads, dim_mlp, dropout):
3         super().__init__()
4         self.self_attn = MultiHeadAttention(
5             in_dim=d_model, qk_dim=d_model,
6             v_dim=d_model, out_dim=d_model,
7             n_heads=n_heads, attn_dropout=dropout,
8         )
9         self.self_attn_dropout = nn.Dropout(dropout)
10        self.self_attn_norm = nn.LayerNorm(d_model)
11
12        self.cross_attn = MultiHeadAttention(
13            in_dim=d_model, qk_dim=d_model,
14            v_dim=d_model, out_dim=d_model,
15            n_heads=n_heads, attn_dropout=dropout,
16        )
17        self.cross_attn_dropout = nn.Dropout(dropout)
18        self.cross_attn_norm = nn.LayerNorm(d_model)
19
20        self.mlp = nn.Sequential(

```

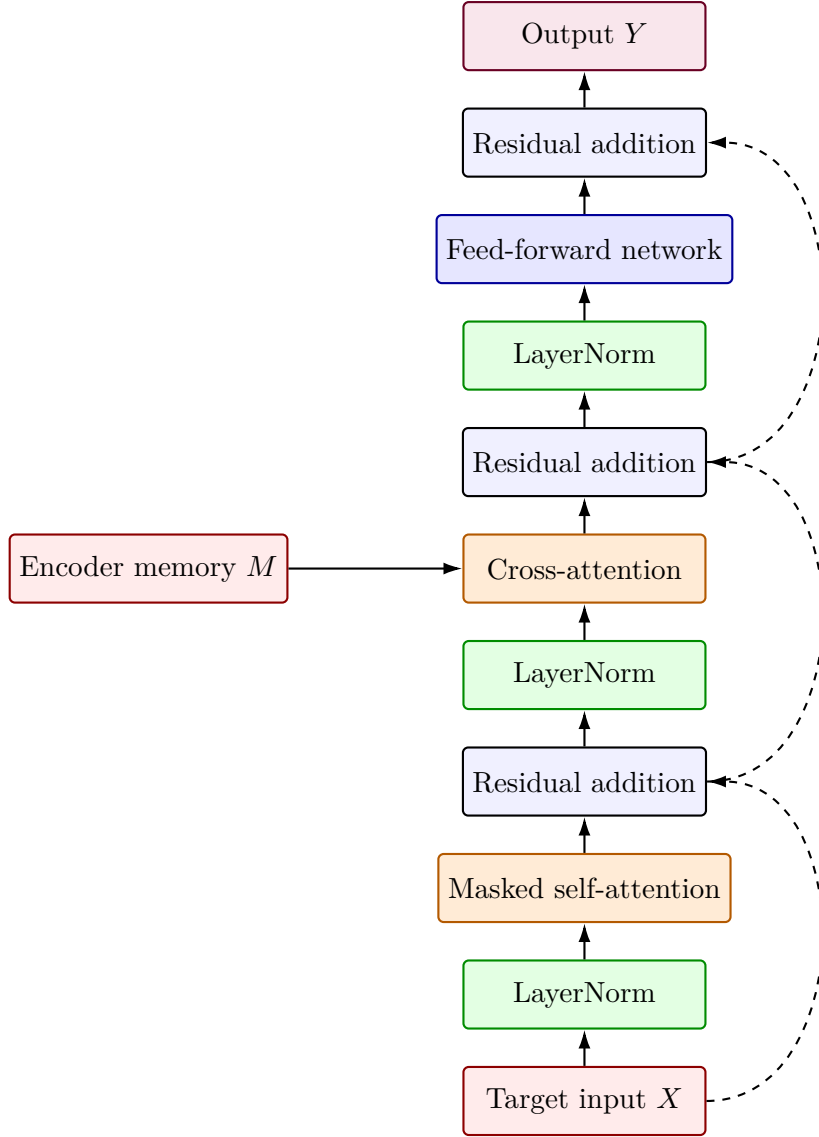


Figure 3: Pre-LayerNorm decoder block with masked self-attention and encoder–decoder cross-attention.


```

21         nn.Linear(d_model, dim_mlp),
22         nn.ReLU(),
23         nn.Linear(dim_mlp, d_model),
24     )
25     self.mlp_dropout = nn.Dropout(dropout)
26     self.mlp_norm = nn.LayerNorm(d_model)
27
28     def forward(self, x, mem, mem_mask=None):
29         _, T, _ = x.shape
30         causal_mask = torch.ones(
31             1, T, T, dtype=torch.bool, device=x.device
32         ).tril()
33
34         qkv = self.self_attn_norm(x)
35         z, _ = self.self_attn(qkv, qkv, qkv,
36                               mask=causal_mask)
37         z = x + self.self_attn_dropout(z)
38
39         cross_query = self.cross_attn_norm(z)
40         if mem_mask is not None:
41             mem_mask = mem_mask.unsqueeze(dim=-2)
42         c, _ = self.cross_attn(
43             cross_query, mem, mem, mask=mem_mask
44         )
45         c = z + self.cross_attn_dropout(c)
46
47         r = self.mlp(self.mlp_norm(c))
48         r = c + self.mlp_dropout(r)
49         return r

```

Listing 4: Pre-LayerNorm decoder block.

6 Token and Position Embeddings

Encoder and decoder blocks accept vector sequences, not discrete token IDs. For vocabulary size V and maximum sequence length $L_{\max}$, define a word embedding matrix $E_w \in \mathbb{R}^{V \times d_{\text{model}}}$ and a positional embedding matrix $E_p \in \mathbb{R}^{L_{\max} \times d_{\text{model}}}$. The representation at position t is

$$e_t = s E_w[x_t] + E_p[t], \quad (19)$$

where the original Transformer used $s = \sqrt{d_{\text{model}}}$ and sinusoidal positions [1]. The implementation below uses learned positional embeddings.

Self-attention is permutation equivariant: permuting the input rows permutes the output rows in the same way. Position information is therefore essential for language sequences.

```

1 class TokenEmbedding(nn.Module):
2     def __init__(self, word_embed_weight, pos_embed_weight,
3                 scale, dropout):
4         super().__init__()
5         max_len, _ = pos_embed_weight.shape
6         self.max_len = max_len
7         self.word_embed_weight = word_embed_weight
8         self.pos_embed_weight = pos_embed_weight
9         self.dropout = nn.Dropout(dropout)
10
11         positions = torch.arange(max_len).unsqueeze(dim=0)
12         self.register_buffer("positions", positions)
13         self.register_buffer(
14             "scale", torch.tensor(float(scale))
15         )

```

```

16
17 def forward(self, x):
18     _, T = x.shape
19     if T > self.max_len:
20         raise RuntimeError(
21             "Sequence length exceeds maximum allowed limit"
22         )
23
24     pos = self.positions[:, :T]
25     word_embed = F.embedding(x, self.word_embed_weight)
26     pos_embed = F.embedding(pos, self.pos_embed_weight)
27     embed = pos_embed + word_embed * self.scale
28     return self.dropout(embed)

```

Listing 5: Token-plus-position embedding layer.

The source and target embedding matrices may be shared when vocabularies are compatible. The target embedding matrix can also be reused as the decoder output projection, a technique known as weight tying [5]. Weight tying reduces parameters and often improves language-model performance, but initialization and scaling must keep the output logits in a numerically useful range.

7 Complete Transformer

The complete encoder-decoder model consists of source and target embeddings, an encoder stack, a decoder stack, final normalizations, and a projection from decoder hidden states to target-vocabulary logits.

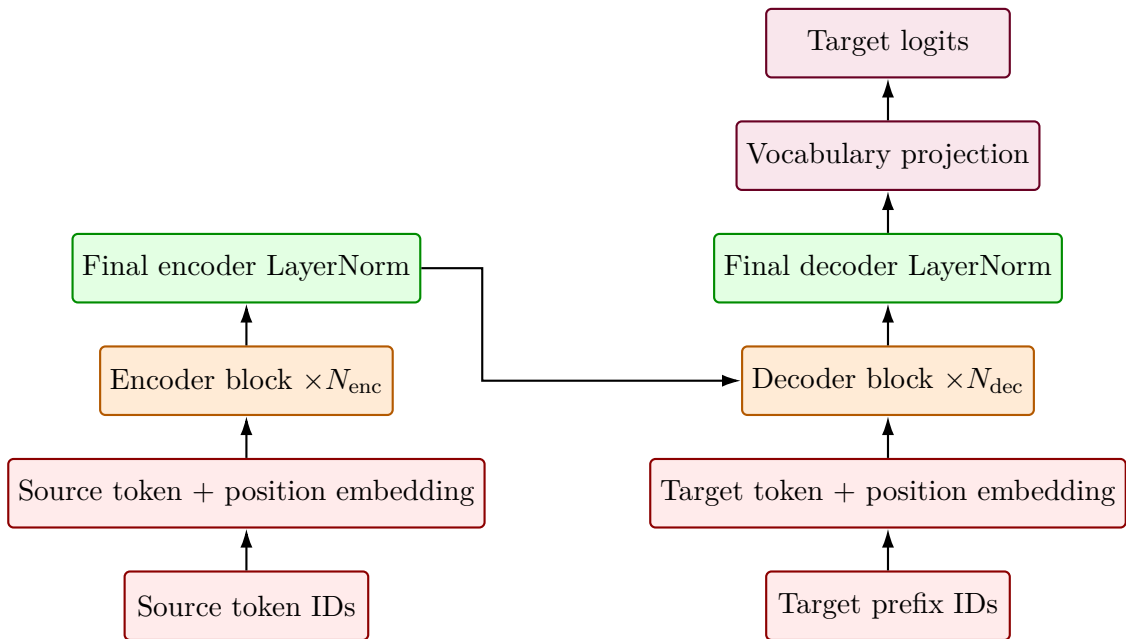


Figure 4: Complete Pre-LayerNorm Transformer. Encoder output is supplied as key-value memory to every decoder block.

```

1 class Transformer(nn.Module):
2     def __init__(self, src_vocab_size, tgt_vocab_size,
3                   max_seq_len, d_model, n_heads,
4                   n_enc, n_dec, dim_mlp, dropout):
5         super().__init__()
6         scale = np.sqrt(d_model)

```

```

7
8     pos_embed = nn.Parameter(
9         torch.randn(max_seq_len, d_model)
10    )
11    src_word_embed = nn.Parameter(
12        torch.randn(src_vocab_size, d_model) / scale
13    )
14
15    if tgt_vocab_size is None:
16        tgt_word_embed = src_word_embed
17    else:
18        tgt_word_embed = nn.Parameter(
19            torch.randn(tgt_vocab_size, d_model) / scale
20        )
21
22    self.src_embed = TokenEmbedding(
23        src_word_embed, pos_embed, scale, dropout
24    )
25    self.tgt_embed = TokenEmbedding(
26        tgt_word_embed, pos_embed, scale, dropout
27    )
28    self.tgt_proj_weight = tgt_word_embed
29
30    self.encoder_stack = nn.ModuleList(
31        EncoderBlock(d_model, n_heads, dim_mlp, dropout)
32        for _ in range(n_enc)
33    )
34    self.enc_norm = nn.LayerNorm(d_model)
35
36    self.decoder_stack = nn.ModuleList(
37        DecoderBlock(d_model, n_heads, dim_mlp, dropout)
38        for _ in range(n_dec)
39    )
40    self.dec_norm = nn.LayerNorm(d_model)

```

Listing 6: Transformer initialization.

```

1 class Transformer(nn.Module):
2     # __init__ as above
3
4     def encode(self, src, src_mask=None):
5         z = self.src_embed(src)
6         for encoder in self.encoder_stack:
7             z = encoder(z, src_mask)
8         return self.enc_norm(z)
9
10    def decode(self, tgt, mem, mem_mask=None):
11        z = self.tgt_embed(tgt)
12        for decoder in self.decoder_stack:
13            z = decoder(z, mem, mem_mask)
14        return self.dec_norm(z)
15
16    def forward(self, src, tgt, src_mask=None):
17        mem = self.encode(src, src_mask)
18        out = self.decode(tgt, mem, src_mask)
19        tgt_scores = F.linear(out, self.tgt_proj_weight)
20        return tgt_scores

```

Listing 7: Encoding, decoding, and teacher-forced forward pass.

For a batch size B , source length T_s , target length T_t , and target vocabulary size V_t , the principal tensor shapes are

Quantity	Shape
Source IDs	$B \times T_s$
Target input IDs	$B \times T_t$
Encoder memory	$B \times T_s \times d_{\text{model}}$
Decoder states	$B \times T_t \times d_{\text{model}}$
Target logits	$B \times T_t \times V_t$

During training, the decoder receives the ground-truth target shifted right. If the target is

$$[\text{BOS}, y_1, y_2, \dots, y_T, \text{EOS}],$$

then the input and output arrays are

$$\text{tgt_in} = [\text{BOS}, y_1, \dots, y_T], \quad (20)$$

$$\text{tgt_out} = [y_1, \dots, y_T, \text{EOS}]. \quad (21)$$

This is teacher forcing: all target positions can be trained in parallel, while the causal mask ensures that position t cannot use later target tokens.

8 Autoregressive Inference

At inference time, the true target sequence is unavailable. The encoder memory is computed once, and the decoder is repeatedly called on the generated prefix. Greedy decoding selects the largest-logit token at each step:

$$\hat{y}_{t+1} = \arg \max_v z_{t,v}. \quad (22)$$

The selected token is appended to the prefix, and decoding continues until an end-of-sequence token is produced or a maximum length is reached.

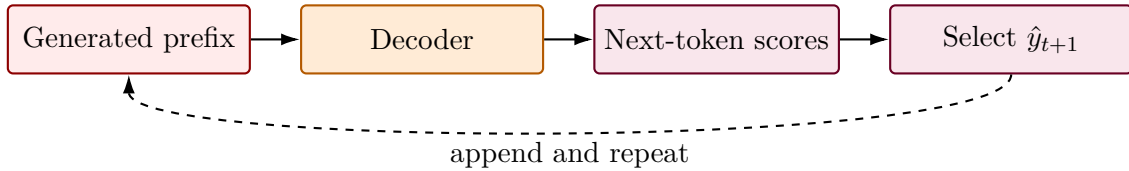


Figure 5: Autoregressive generation is an external loop around the non-recurrent Transformer decoder.

```

1 class Transformer(nn.Module):
2     # other methods as above
3
4     @torch.no_grad()
5     def greedy_decode(self, src, src_mask,
6                       bos_idx, eos_idx, max_len=80):
7         B = src.shape[0]
8         done = torch.zeros(B, dtype=torch.bool,
9                             device=src.device)
10        was_training = self.training
11        self.eval()
12
13        tgt = torch.full(
14            (B, 1), bos_idx,
15            dtype=torch.long, device=src.device
16        )
17        mem = self.encode(src, src_mask)
18

```

```

19     for _ in range(max_len - 1):
20         out = self.decode(tgt, mem, mem_mask=src_mask)
21         scores = F.linear(out, self.tgt_proj_weight)
22         next_idx = scores[:, -1].argmax(dim=-1, keepdim=True)
23         tgt = torch.cat((tgt, next_idx), dim=1)
24         done |= next_idx.squeeze(-1).eq(eos_idx)
25         if done.all():
26             break
27
28     if was_training:
29         self.train()
30     return tgt

```

Listing 8: Greedy autoregressive decoding.

Beginning- and end-of-sequence IDs are conventional control symbols rather than intrinsic requirements of the network. The model can be prompted with other vocabulary items, but termination behavior is most reliable when training explicitly associates a designated end token with sequence completion.

9 Experiment I: Reversing a Sequence

A simple verification task is to reverse variable-length integer sequences. Each target is the reversed source surrounded by beginning- and end-of-sequence markers.

```

1  from random import randint
2  from torch.nn.utils.rnn import pad_sequence
3
4
5  device = torch.device(
6      "cuda" if torch.cuda.is_available() else "cpu"
7  )
8  bos_idx, eos_idx, pad_idx = 1, 2, 0
9  vocab_size, src_len = 100, 16
10
11
12  def collate_reverse(batch):
13      src = pad_sequence(
14          batch, batch_first=True, padding_value=pad_idx
15      )
16      tgt = pad_sequence(
17          [torch.tensor(
18              [bos_idx] + x.flip(0).tolist() + [eos_idx]
19          ) for x in batch],
20          batch_first=True,
21          padding_value=pad_idx,
22      )
23      return src, tgt
24
25
26  data_loader = data.DataLoader(
27      dataset=[
28          torch.randint(
29              3, vocab_size,
30              (randint(src_len // 2, src_len),)
31          )
32          for _ in range(50_000)
33      ],
34      batch_size=128,
35      shuffle=True,
36      drop_last=True,
37      collate_fn=collate_reverse,

```

38)

Listing 9: Synthetic reversal dataset.

The model is deliberately small: two encoder blocks, two decoder blocks, $d_{\text{model}} = 64$, two heads, and an MLP width of 128. With a shared 100-token embedding table and 32 learned positions, this implementation has approximately 175,000 trainable parameters.

```

1 transformer = Transformer(
2     src_vocab_size=vocab_size,
3     tgt_vocab_size=None,
4     max_seq_len=32,
5     d_model=64,
6     n_heads=2,
7     n_enc=2,
8     n_dec=2,
9     dim_mlp=128,
10    dropout=0.1,
11 ).to(device)
12
13 optim = torch.optim.AdamW(
14     transformer.parameters(),
15     lr=1e-3,
16     weight_decay=1e-4,
17 )
18
19 for _ in range(10):
20     for src, tgt in data_loader:
21         src, tgt = src.to(device), tgt.to(device)
22         tgt_in, tgt_out = tgt[:, :-1], tgt[:, 1:]
23         logits = transformer(src, tgt_in, src != pad_idx)
24         loss = F.cross_entropy(
25             logits.permute(0, 2, 1),
26             tgt_out,
27             ignore_index=pad_idx,
28         )
29
30         optim.zero_grad()
31         loss.backward()
32         torch.nn.utils.clip_grad_norm_(
33             transformer.parameters(), 1.0
34         )
35         optim.step()
36
37 x = torch.tensor(
38     [3, 5, 8, 13, 21, 34, 55, 89]
39 ).unsqueeze(0).to(device)
40 y = transformer.greedy_decode(
41     x, None, bos_idx, eos_idx, max_len=32
42 )
43 print(y)
44 # [1, 89, 55, 34, 21, 13, 8, 5, 3, 2]
```

Listing 10: Training and evaluating the reversal model.

The example highlights the shifted-target convention: the end token appears in the target output but not in the corresponding decoder input. Padding is excluded from the loss and from source attention through a Boolean mask.

10 Experiment II: German–English Translation

A more realistic test uses the Multi30K German–English dataset [6]. The following compact pipeline tokenizes both languages, constructs vocabularies with special symbols, and pads each batch.

```
1 from collections import Counter
2 from torch.nn.utils.rnn import pad_sequence
3 from torchtext.datasets import Multi30k
4 from torchtext.data.utils import get_tokenizer
5 from torchtext.vocab import vocab
6
7 train_data = Multi30k(root="datasets", split="train")
8 train_data = [
9     (src, tgt) for src, tgt in train_data if len(src) > 0
10 ]
11
12 UNK, PAD, BOS, EOS = (
13     "<UNK>", "<PAD>", "<START>", "<END>"
14 )
15 tokenizer = get_tokenizer("basic_english")
16
17 de_counter, en_counter = Counter(), Counter()
18 for src, tgt in train_data:
19     de_counter.update(tokenizer(src))
20     en_counter.update(tokenizer(tgt))
21
22 de_vocab = vocab(
23     de_counter, specials=[UNK, PAD, BOS, EOS]
24 )
25 en_vocab = vocab(
26     en_counter, specials=[UNK, PAD, BOS, EOS]
27 )
28 de_vocab.set_default_index(de_vocab[UNK])
29 en_vocab.set_default_index(en_vocab[UNK])
30
31 pad_idx = de_vocab[PAD]
32 assert en_vocab[PAD] == pad_idx
33
34
35 def collate_translation(batch):
36     src = pad_sequence(
37         [torch.tensor(de_vocab(tokenizer(x)))
38          for x, _ in batch],
39         batch_first=True,
40         padding_value=pad_idx,
41     )
42     tgt = pad_sequence(
43         [torch.tensor(
44             en_vocab([BOS] + tokenizer(y) + [EOS])
45         ) for _, y in batch],
46         batch_first=True,
47         padding_value=pad_idx,
48     )
49     return src, tgt
50
51 train_loader = data.DataLoader(
52     dataset=train_data,
53     batch_size=128,
54     shuffle=True,
55     drop_last=True,
56     collate_fn=collate_translation,
57     num_workers=4,
```

58)

Listing 11: Preparing Multi30K data.

```

1 transformer = Transformer(
2     src_vocab_size=len(de_vocab),
3     tgt_vocab_size=len(en_vocab),
4     max_seq_len=256,
5     d_model=256,
6     n_heads=8,
7     n_enc=4,
8     n_dec=4,
9     dim_mlp=512,
10    dropout=0.1,
11 ).to(device)
12
13 optim = torch.optim.AdamW(
14     transformer.parameters(),
15     lr=1e-4,
16     weight_decay=1e-4,
17 )
18
19 for _ in range(30):
20     for src, tgt in train_loader:
21         src, tgt = src.to(device), tgt.to(device)
22         tgt_in, tgt_out = tgt[:, :-1], tgt[:, 1:]
23         src_mask = src != pad_idx
24
25         logits = transformer(src, tgt_in, src_mask)
26         loss = F.cross_entropy(
27             logits.permute(0, 2, 1),
28             tgt_out,
29             ignore_index=pad_idx,
30         )
31
32         optim.zero_grad()
33         loss.backward()
34         torch.nn.utils.clip_grad_norm_(
35             transformer.parameters(), 1.0
36         )
37         optim.step()

```

Listing 12: Training the translation model.

A representative source sentence and greedy translation are

```

1 sent = [
2     "zwei", "frauen", "spazieren", "und",
3     "lachen", "im", "park", "."
4 ]
5 x = torch.tensor(de_vocab(sent)).unsqueeze(0).to(device)
6 y = transformer.greedy_decode(
7     x, None, en_vocab[BOS], en_vocab[EOS]
8 )
9 print(en_vocab.lookup_tokens(y[0].tolist()))
10 # ['<START>', 'two', 'women', 'are', 'walking',
11 #  'and', 'laughing', 'in', 'the', 'park', '.', '<END>']

```

Listing 13: Example translation.

The example is intentionally modest and is not intended as a competitive translation benchmark. Its purpose is to verify that the same implementation supports a genuine source–target language task.

11 Batching by Sequence Length

Padding is computationally wasteful because attention operates on the padded rectangular batch. Grouping examples of similar lengths reduces this overhead. A practical compromise is to shuffle indices globally, divide them into moderately large pools, sort each pool by sequence length, and then form batches from adjacent sorted examples.

```
1 class BatchSampler:
2     def __init__(self, lengths, batch_size):
3         self.lengths = lengths
4         self.batch_size = batch_size
5
6     def __iter__(self):
7         size = len(self.lengths)
8         indices = list(range(size))
9         random.shuffle(indices)
10
11         step = 100 * self.batch_size
12         for i in range(0, size, step):
13             pool = indices[i:i + step]
14             pool = sorted(
15                 pool, key=lambda idx: self.lengths[idx]
16             )
17
18             for j in range(0, len(pool), self.batch_size):
19                 if j + self.batch_size > len(pool):
20                     break # drop_last=True
21                 yield pool[j:j + self.batch_size]
22
23     def __len__(self):
24         return len(self.lengths) // self.batch_size
```

Listing 14: Length-aware batch sampler.

The sampler is supplied through the `batch_sampler` argument rather than through the ordinary batch-size and shuffle arguments. In the reported experiment, the average number of padding tokens per sequence decreased from approximately 15.25 to 3.62.

```
1 lengths = [len(src) for src, _ in train_data]
2 batch_size = 128
3
4 train_loader = data.DataLoader(
5     dataset=train_data,
6     batch_sampler=BatchSampler(lengths, batch_size),
7     collate_fn=collate_translation,
8     num_workers=4,
9 )
10
11 avg_pads = sum(
12     (src == pad_idx).sum() / src.shape[0]
13     for src, _ in train_loader
14 ) / len(train_loader)
```

Listing 15: Using the length-aware sampler.

12 Discussion

Several design choices interact:

- **Attention and parallelism.** Unlike recurrent unrolling, attention computes all query positions in parallel during training.

- **Multi-head factorization.** Splitting a fixed model width across heads creates multiple learned relation spaces without increasing the dominant projection dimension.
- **Residual pathways and normalization.** Residual additions preserve a direct information and gradient path. Pre-LayerNorm usually improves optimization stability, while final normalization restores controlled output scale.
- **Feed-forward sublayers.** The position-wise MLP adds nonlinearity and feature transformation after attention mixes information across positions. Other domains may substitute different local transformations; for example, FastSpeech uses convolutional sublayers in its feed-forward Transformer blocks [3].
- **Weight tying.** Sharing target embeddings and output projection reduces parameters and can improve generalization, provided initialization and scaling remain appropriate.
- **Training versus generation.** Teacher forcing exposes the entire shifted target during training, whereas inference feeds previous model predictions back into an external autoregressive loop.

13 Conclusion

The Transformer can be understood as a composition of a few precise operations: learned token and position representations, scaled dot-product attention, head concatenation and output projection, position-wise nonlinear processing, residual connections, and normalization. The encoder turns a source sequence into contextual memory; the decoder combines causal target self-attention with queries into that memory. The accompanying implementation makes these abstractions explicit and demonstrates them on both a controlled reversal task and a small machine-translation example. Practical performance depends not only on architecture but also on masking, initialization, target shifting, gradient control, and efficient batching.

References

- [1] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin, “Attention Is All You Need,” *Advances in Neural Information Processing Systems*, vol. 30, 2017.
- [2] A. M. Rush, “The Annotated Transformer,” 2018. <https://nlp.seas.harvard.edu/annotated-transformer/>
- [3] Y. Ren, Y. Ruan, X. Tan, T. Qin, S. Zhao, Z. Zhao, and T.-Y. Liu, “FastSpeech: Fast, Robust and Controllable Text to Speech,” *Advances in Neural Information Processing Systems*, vol. 32, 2019.
- [4] R. Xiong, Y. Yang, D. He, K. Zheng, S. Zheng, C. Xing, H. Zhang, Y. Lan, L. Wang, and T.-Y. Liu, “On Layer Normalization in the Transformer Architecture,” *Proceedings of the 37th International Conference on Machine Learning*, 2020.
- [5] O. Press and L. Wolf, “Using the Output Embedding to Improve Language Models,” *Proceedings of the 15th Conference of the European Chapter of the Association for Computational Linguistics*, 2017.
- [6] D. Elliott, S. Frank, K. Sima’an, and L. Specia, “Multi30K: Multilingual English–German Image Descriptions,” *Proceedings of the 5th Workshop on Vision and Language*, 2016.